

Centro Federal de Educação Tecnológica de Minas Gerais

Árvores B+

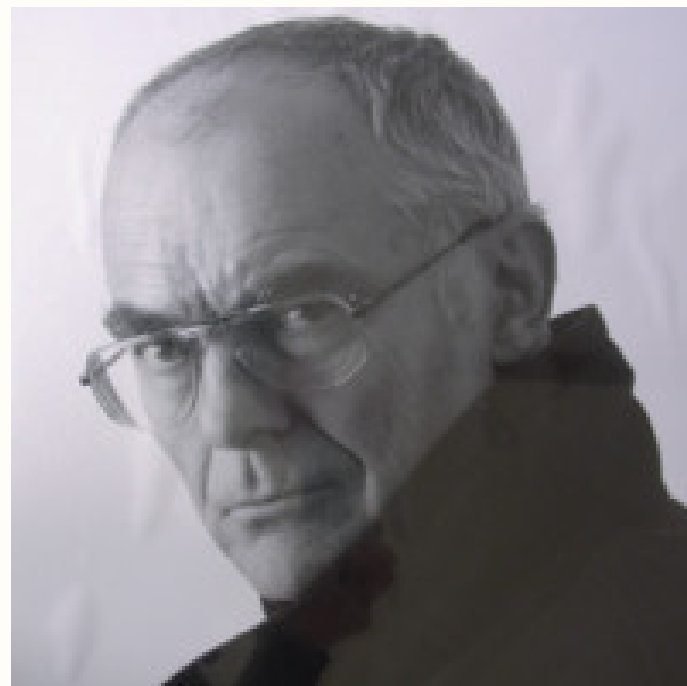
Gabriel Gonçalves de Castro

Sumário:

- Criação - 1
- Estrutura - 4
- Propriedades - 9
- Operações Fundamentais e Implementação - 17
 - Busca - 18
 - Inserção - 20
 - Remoção - 40
 - Busca em Intervalo (Range Query) - 60
- Aplicações - 62
- Exemplos - 66
- Referências - 80

Criação

Criação:



Rudolf Bayer

Acta Informatica 1, 173–189 (1972)
© by Springer-Verlag 1972

Organization and Maintenance of Large Ordered Indexes

R. BAYER and E. MCCREIGHT

Received September 29, 1971

Summary. Organization and maintenance of an index for a dynamic random access file is considered. It is assumed that the index must be kept on some pseudo random access backup store like a disc or a drum. The index organization described allows retrieval, insertion, and deletion of keys in time proportional to $\log_k I$ where I is the size of the index and k is a device dependent natural number such that the performance of the scheme becomes near optimal. Storage utilization is at least 50% but generally much higher. The pages of the index are organized in a special data-structure, so-called *B-trees*. The scheme is analyzed, performance bounds are obtained, and a near optimal k is computed. Experiments have been performed with indexes up to 100 000 keys. An index of size 15 000 (100 000) can be maintained with an average of 9 (at least 4) transactions per second on an IBM 360/44 with a 2311 disc.

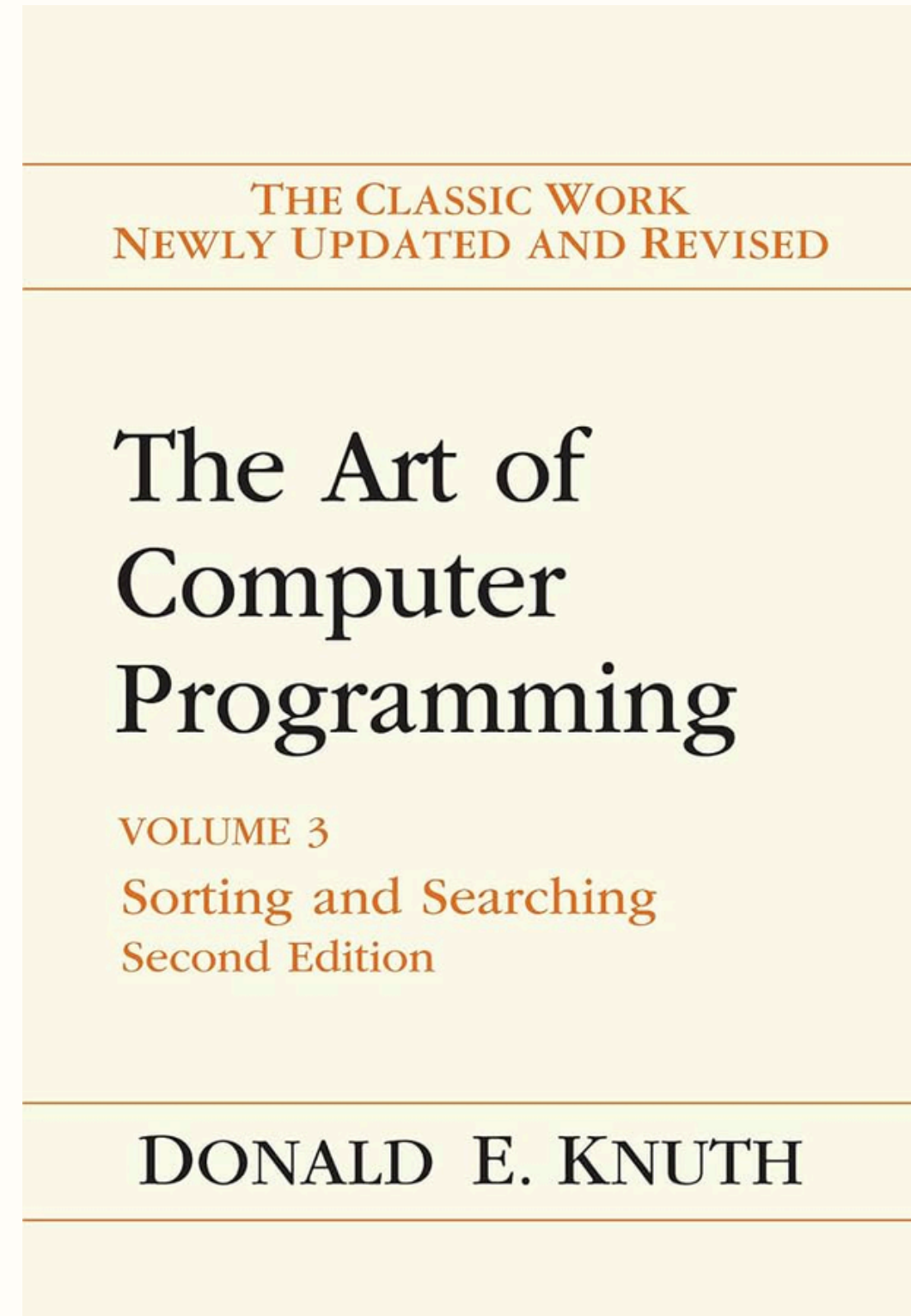
1. Introduction

In this paper we consider the problem of organizing and maintaining an index for a dynamically changing random access file. By an *index* we mean a collection of index elements which are pairs (x, α) of fixed size physically adjacent



Edward M. Macreight

Criação:



Estrutura

Estrutura:

Seja T uma árvore B+ composta por nós, sendo $T.raiz$ o nó raiz.

Estrutura:

Seja T uma árvore B+ composta por nós, sendo $T.raiz$ o nó raiz.

Todo nó x pertencente a T possui os seguintes atributos básicos:

- $x.n$: número de chaves armazenadas em x .
- $x.chave_1 \leq x.chave_2 \leq \dots \leq x.chave_{x.n}$: as chaves armazenadas em ordem crescente.
- $x.folha$: um valor booleano que indica se x é um nó folha.

Estrutura:

Seja T uma árvore B+ composta por nós, sendo $T.raiz$ o nó raiz.

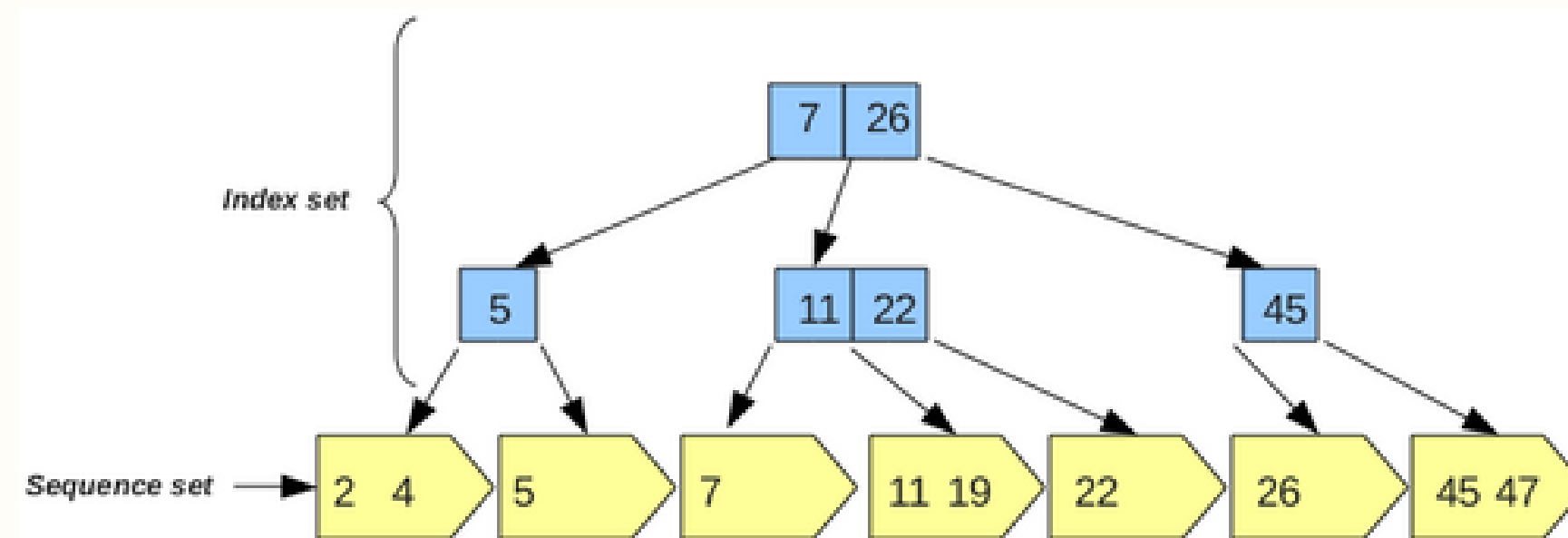
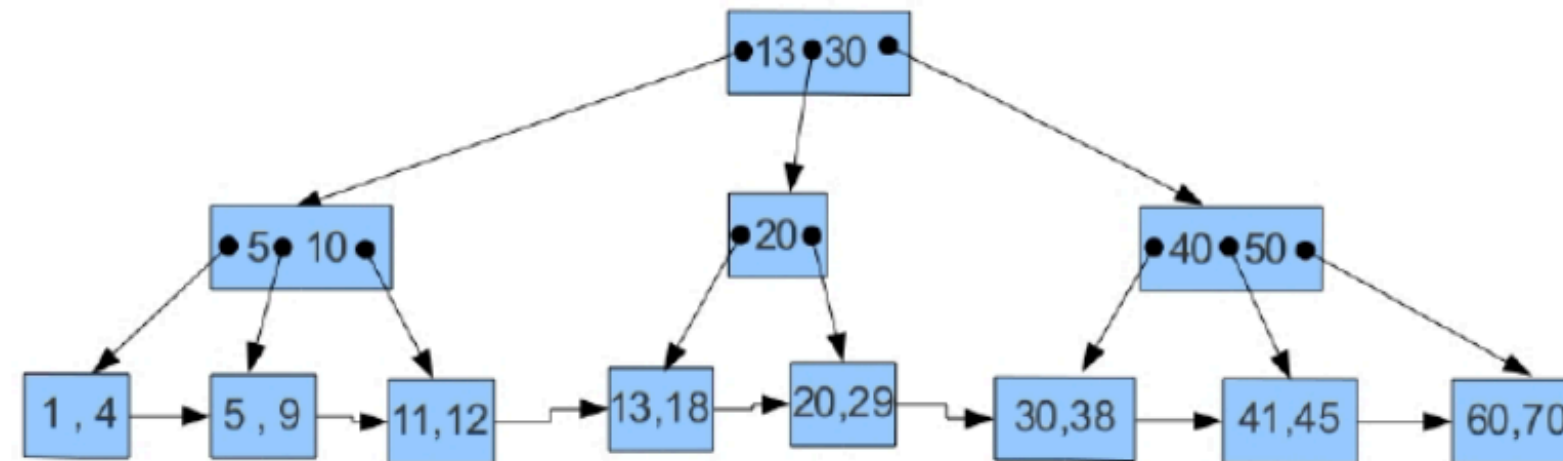
Todo nó x pertencente a T possui os seguintes atributos básicos:

- $x.n$: número de chaves armazenadas em x .
- $x.chave_1 \leq x.chave_2 \leq \dots \leq x.chave_{x.n}$: as chaves armazenadas em ordem crescente.
- $x.folha$: um valor booleano que indica se x é um nó folha.

A estrutura complementar de x depende do seu tipo:

- Caso x seja um nó interno ($x.folha = \text{FALSO}$): Possui $x.n+1$ ponteiros para seus filhos, denotados por $x.filho_1, x.filho_2, \dots, x.filho_{x.n+1}$.
- Caso x seja um nó folha ($x.folha = \text{VERDADEIRO}$): Possui um ponteiro $x.proximo$ que aponta para o próximo nó folha à direita (lista ligada). Também possui $x.n$ dados armazenados, referente as chaves.

Estrutura:



Propriedades

Estrutura:

Uma Arvore B+ deve satisfazer as seguintes condições de balanceamento:

Estrutura:

Uma Arvore B+ deve satisfazer as seguintes condições de balanceamento:

- Todo caminho do nó raiz até um nó folha qualquer deve possuir o mesmo tamanho;

Estrutura:

Uma Arvore B+ deve satisfazer as seguintes condições de balanceamento:

- Todo caminho do nó raiz até um nó folha qualquer deve possuir o mesmo tamanho;
- Os nós possuem limites superiores e inferiores para o número de chaves que podem conter.

Estrutura:

Uma Arvore B+ deve satisfazer as seguintes condições de balanceamento:

- Todo caminho do no raiz até um nó folha qualquer deve possuir o mesmo tamanho;
- Os nós possuem limites superiores e inferiores para o número de chaves que podem conter.

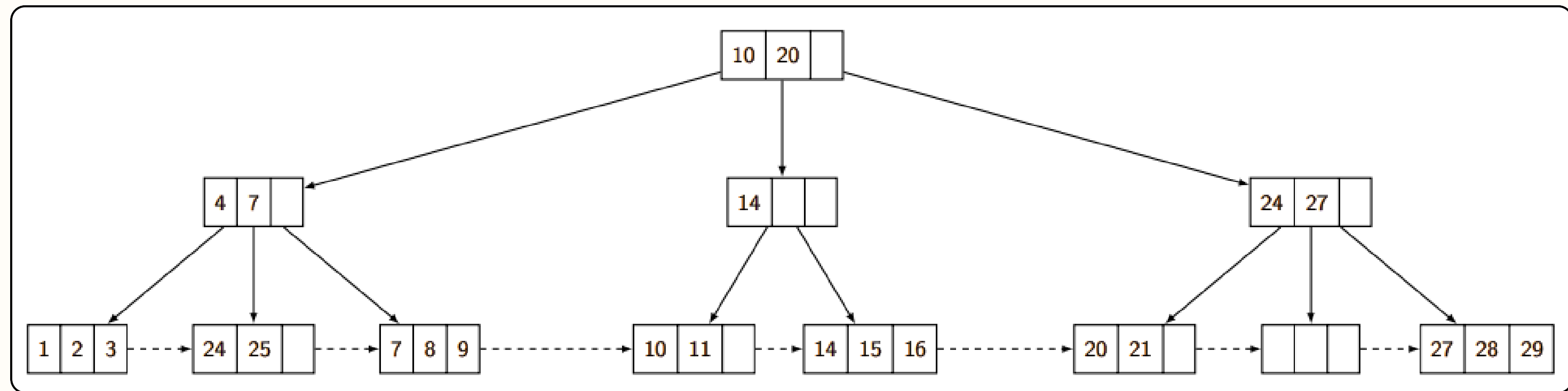
$$t \geq 2$$

Mínimo: $t - 1$

Máximo: $2t - 1$

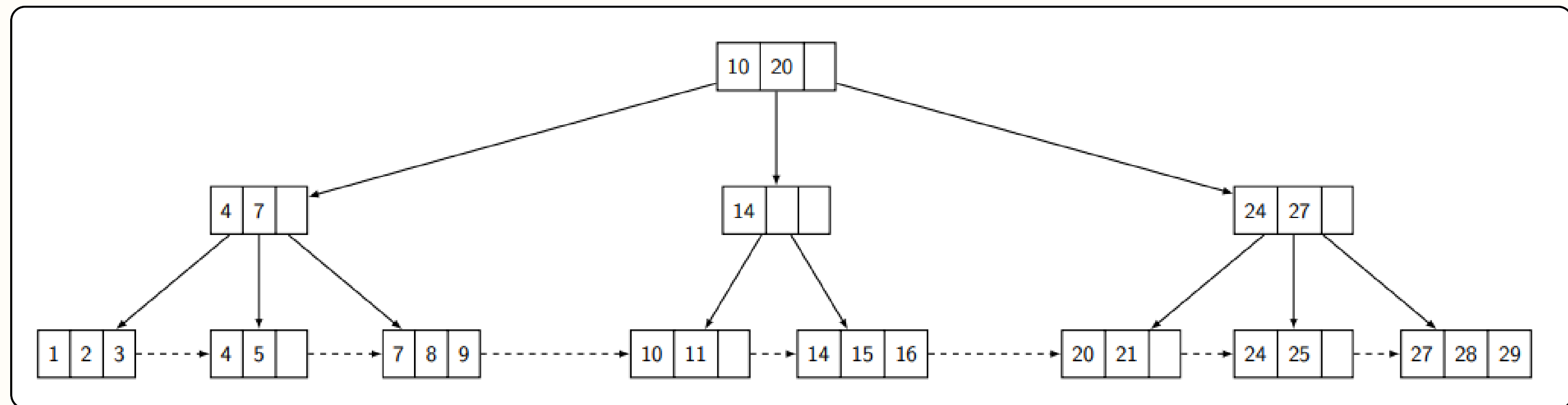
Estrutura:

$t = 2$



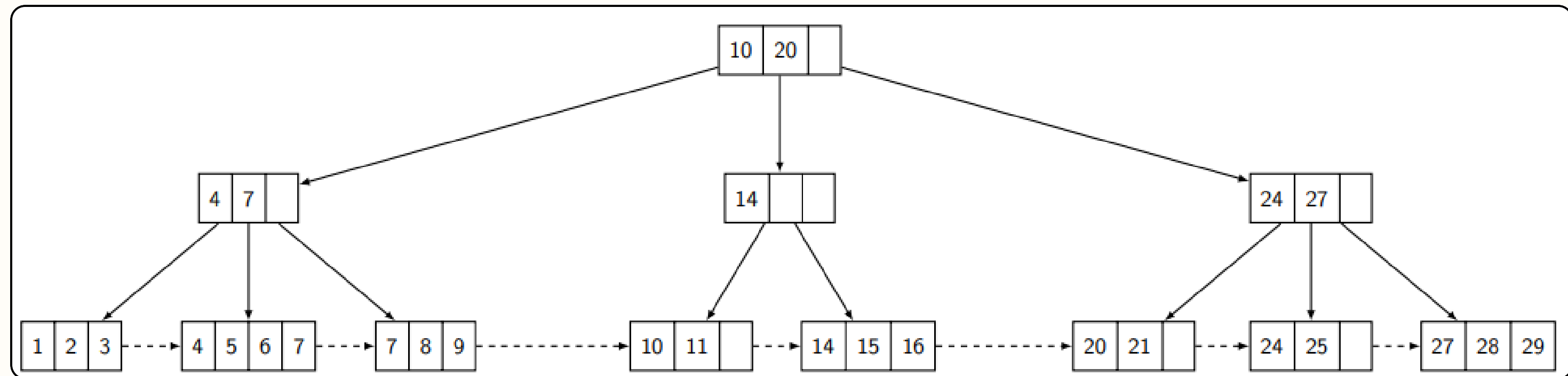
Estrutura:

$t = 2$



Estrutura:

$t = 2$

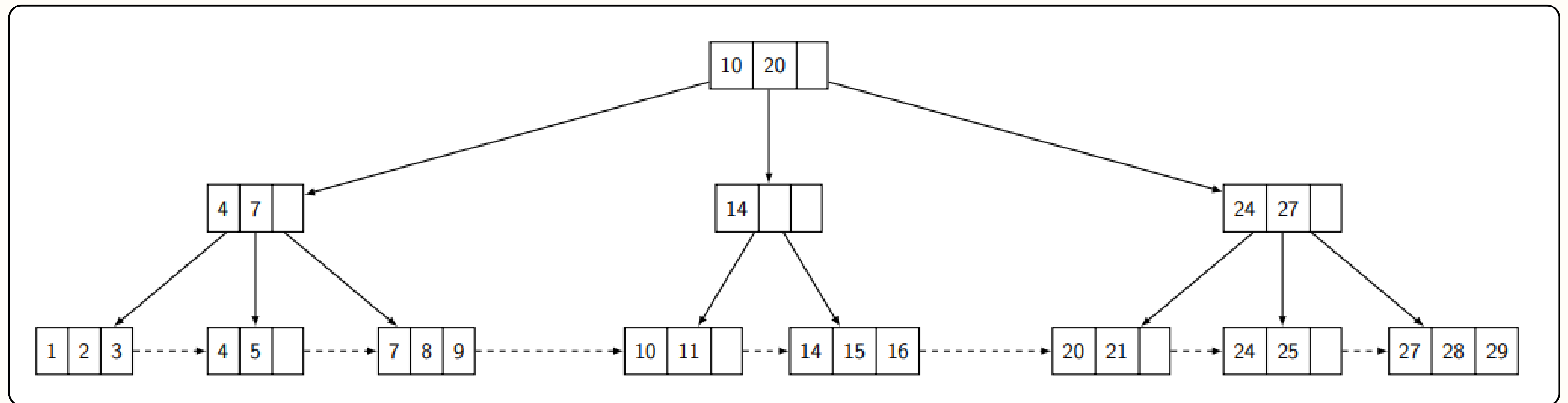


Operações Fundamentais e Implementações

Busca

Busca:

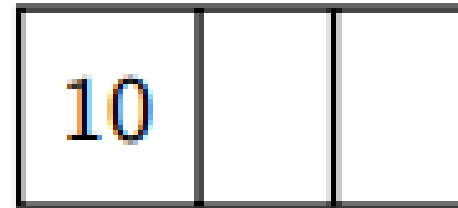
Vamos buscar o valor **21**



Inserção

Inserção:

Inserindo o valor **10**



Inserção:

Inserindo o valor **20**

10		
----	--	--

Insertão:

Inserindo o valor **20**

10	20	
----	----	--

Insertão:

Inserindo o valor **30**

10	20	
----	----	--

Inserção:

Inserindo o valor **30**

10	20	30
----	----	----

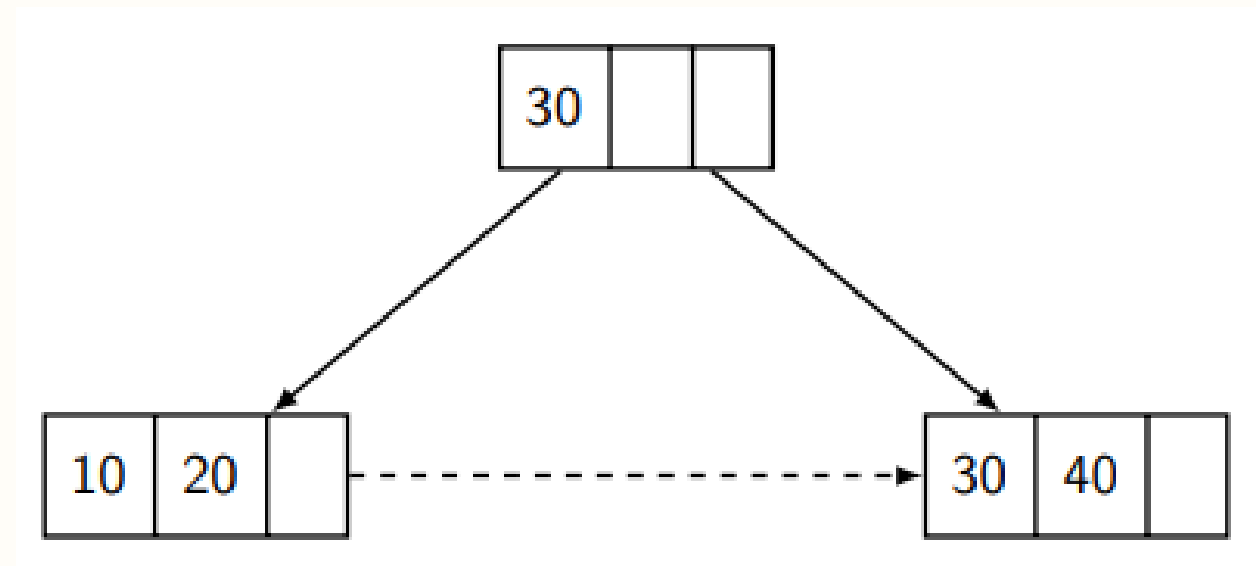
Inserção:

Inserindo o valor **40**

10	20	30
----	----	----

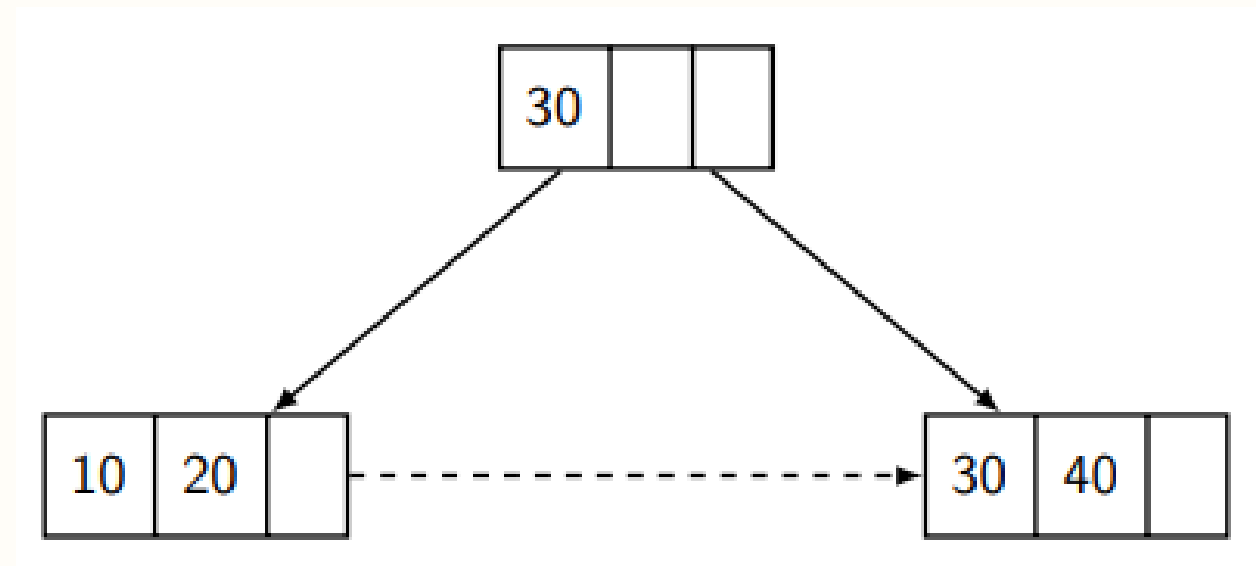
Inserção:

Inserindo o valor **40** (split)



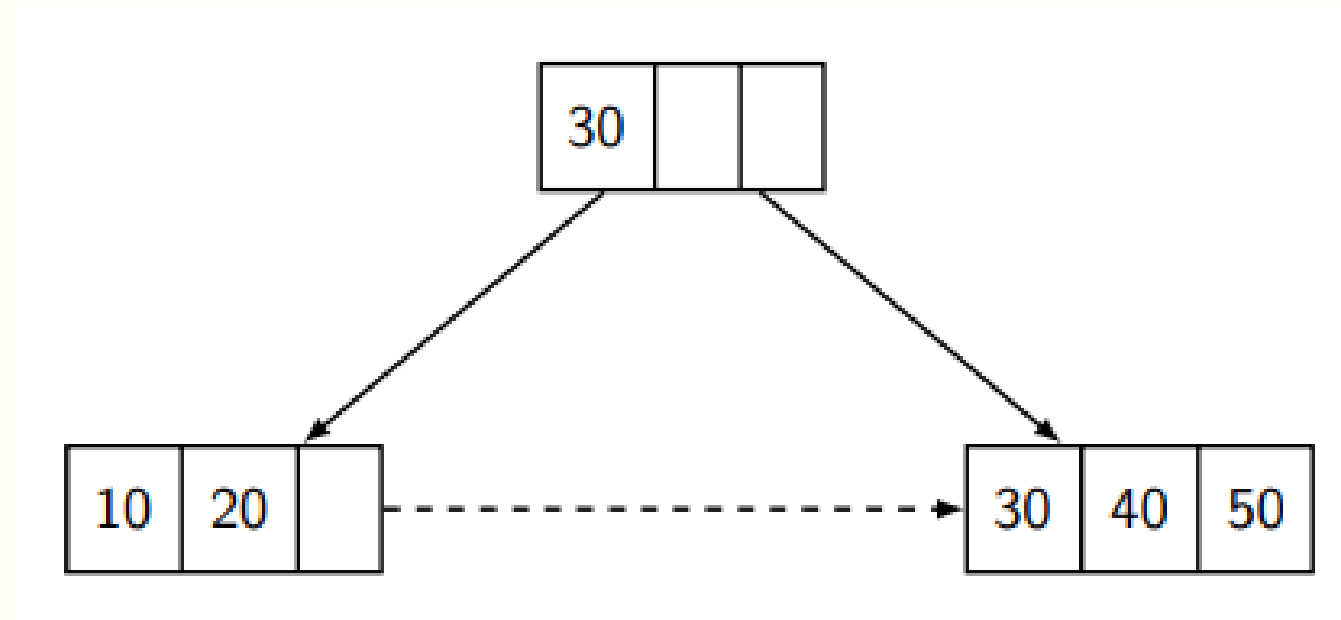
Inserção:

Inserindo o valor **50**



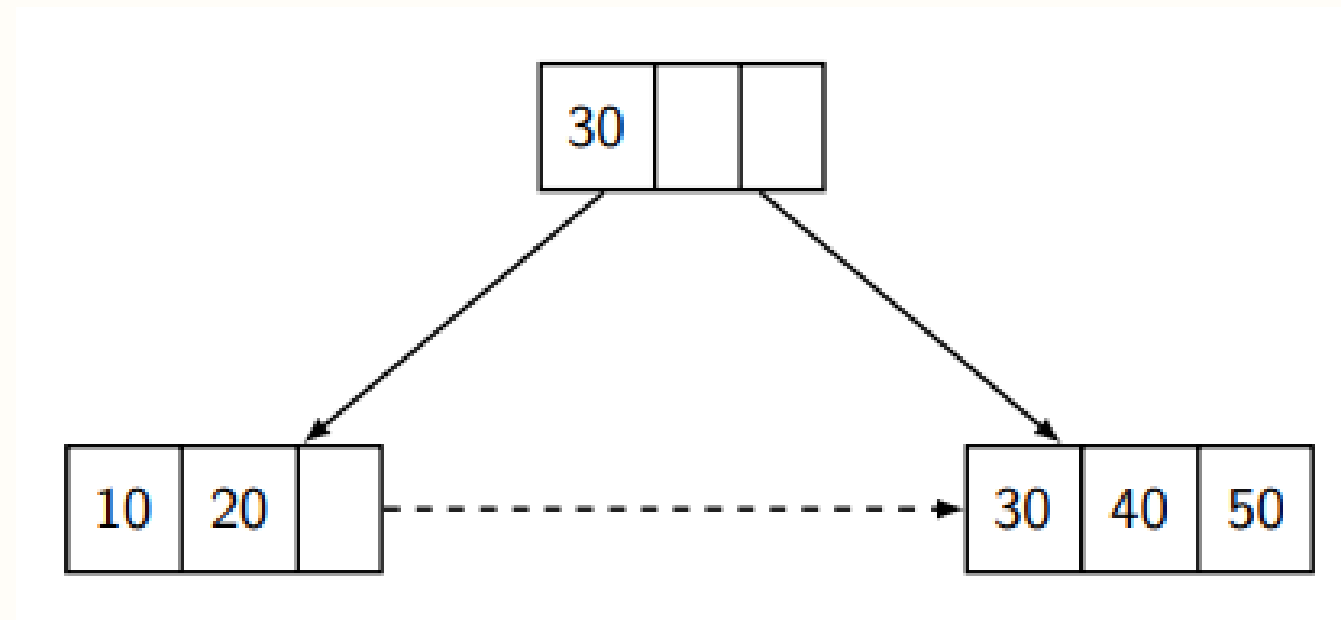
Inserção:

Inserindo o valor **50**



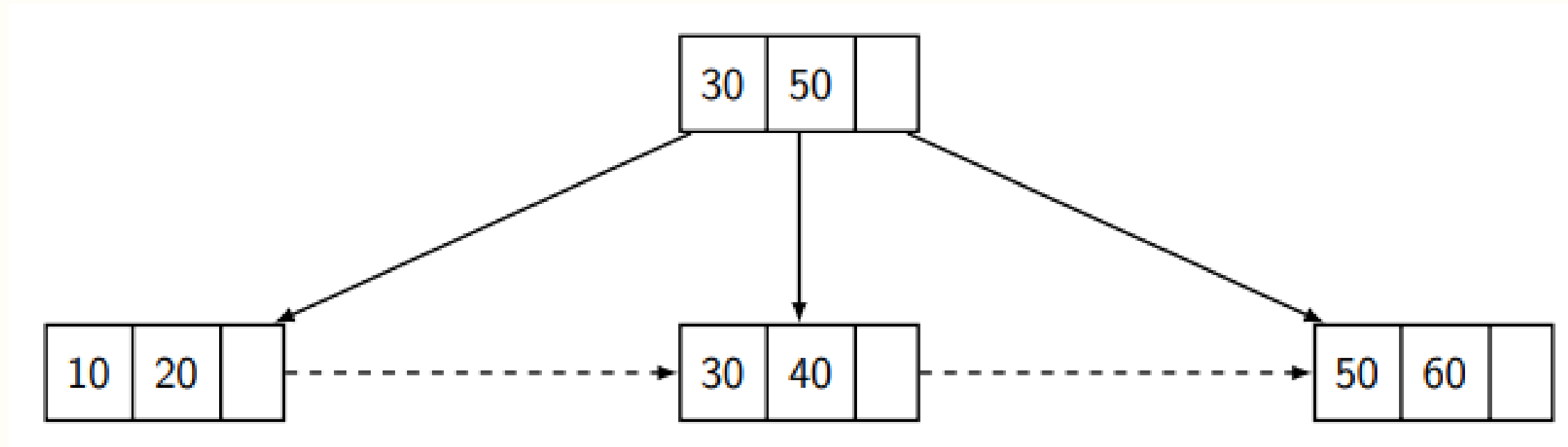
Inserção:

Inserindo o valor **60**



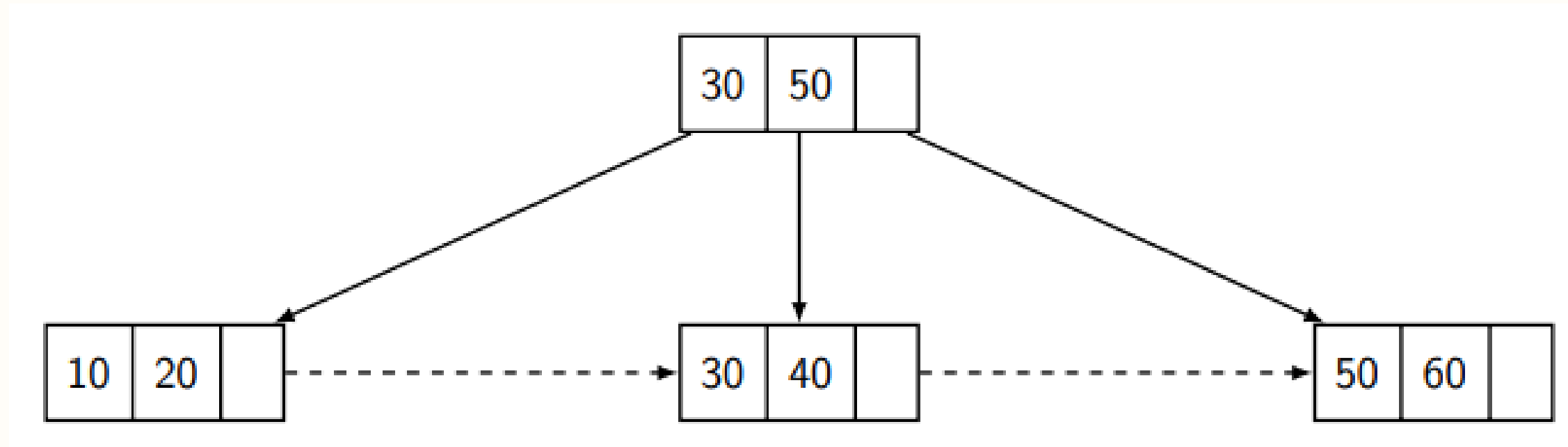
Inserção:

Inserindo o valor **60** (split)



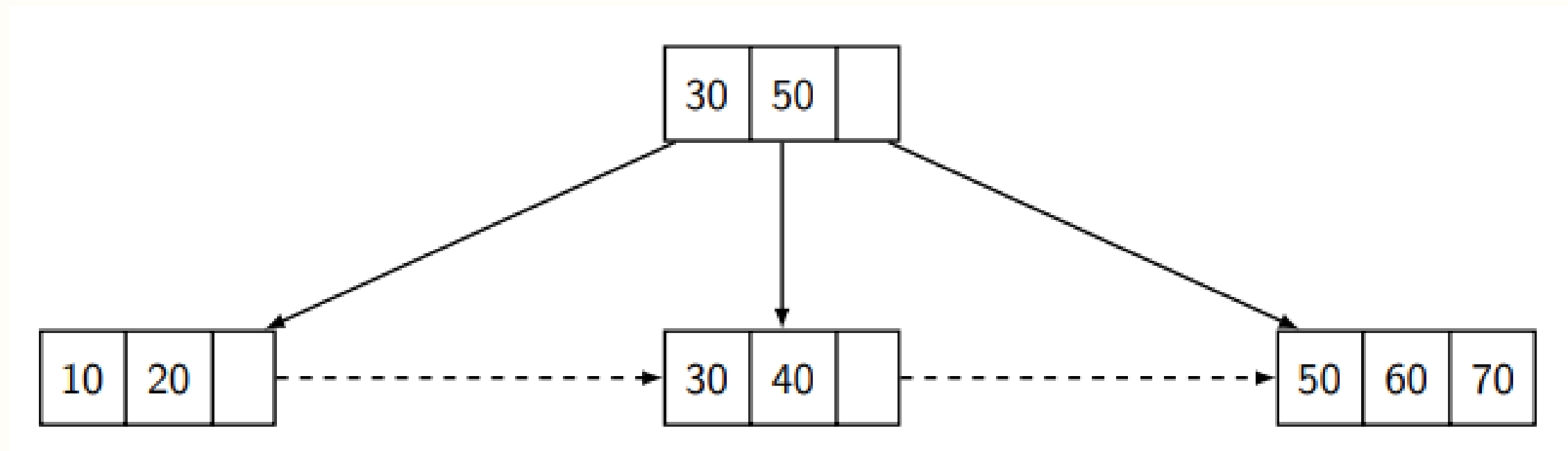
Inserção:

Inserindo o valor **70**



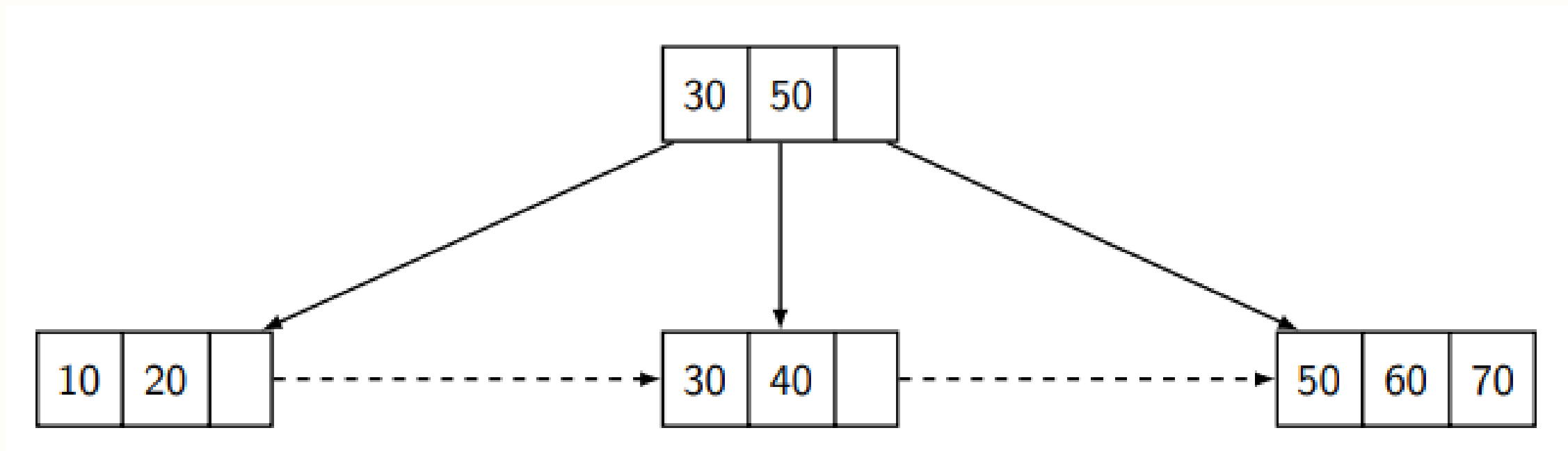
Inserção:

Inserindo o valor **70**



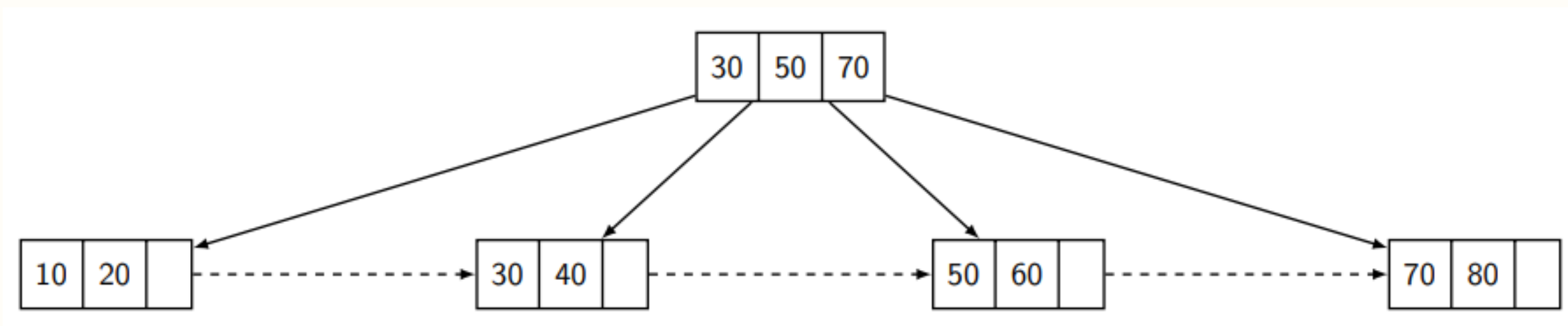
Inserção:

Inserindo o valor **80**



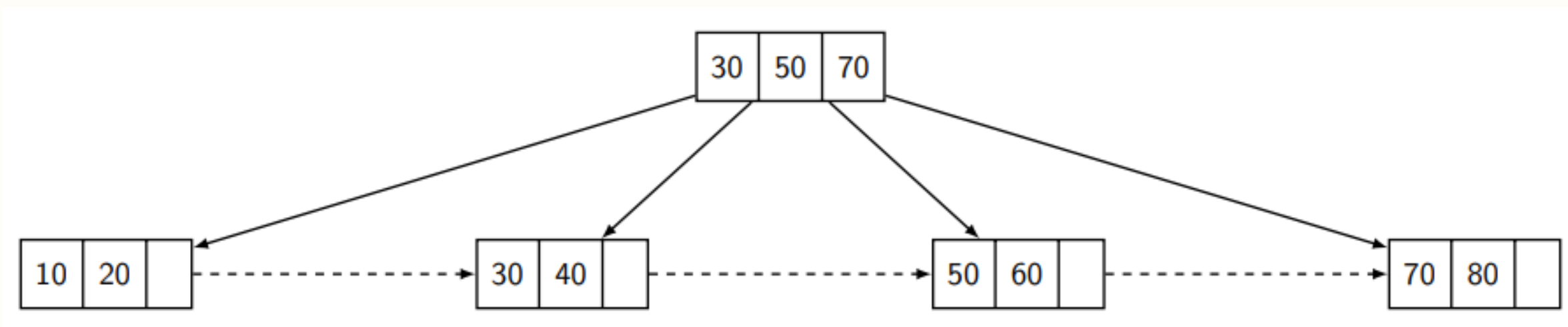
Inserção:

Inserindo o valor **80** (split)



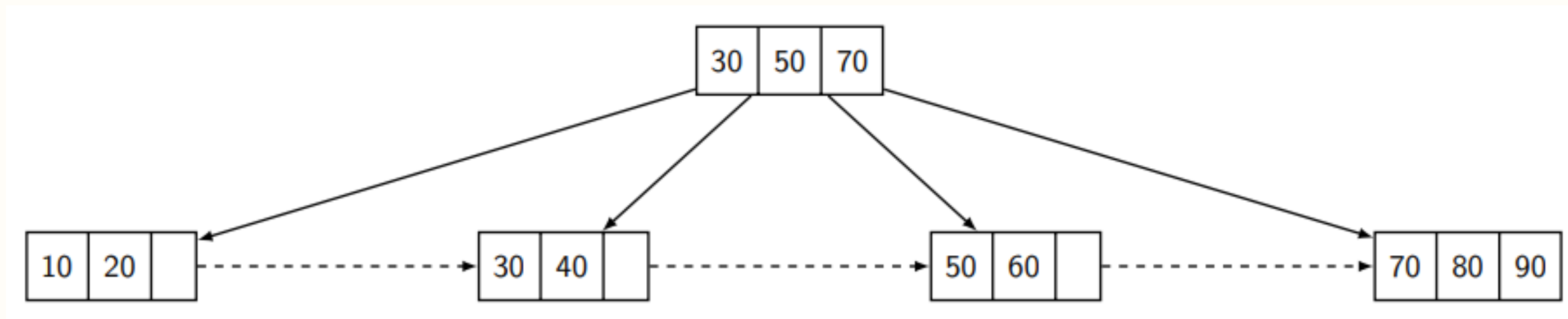
Insertão:

Inserindo o valor **80**



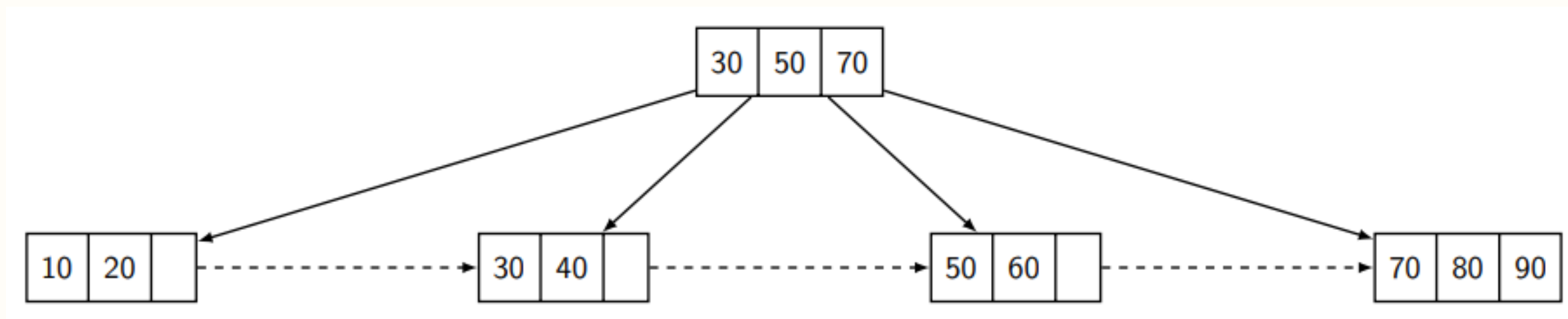
Inserção:

Inserindo o valor **90**



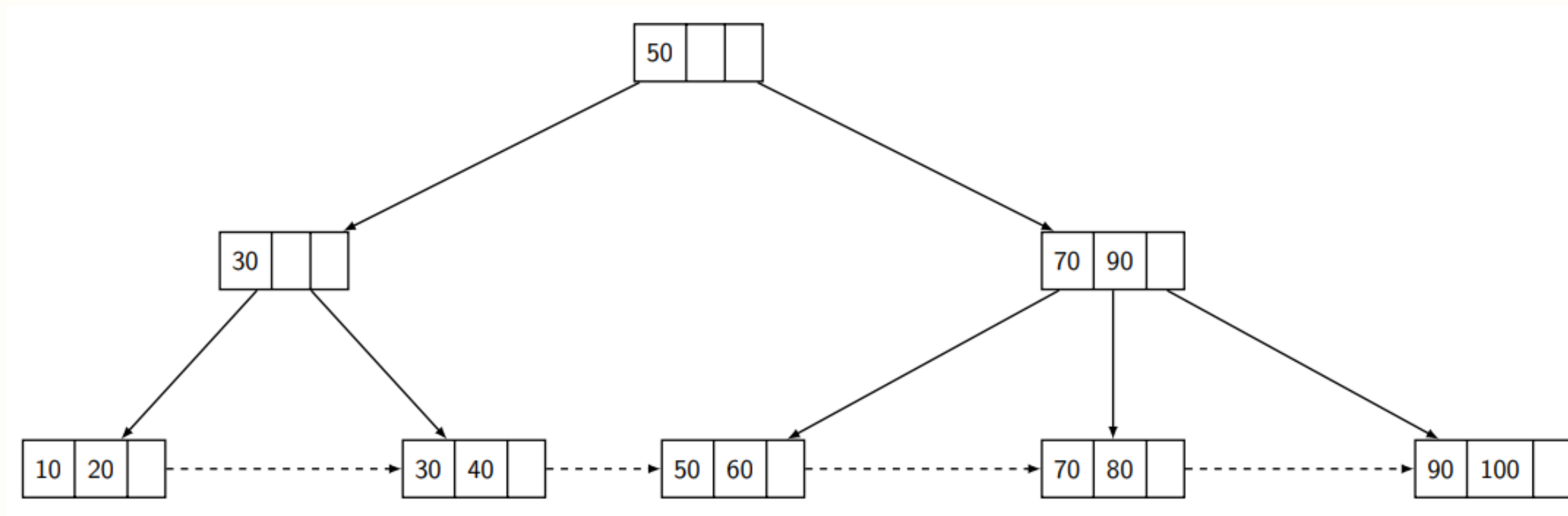
Inserção:

Inserindo o valor **100**



Insertão:

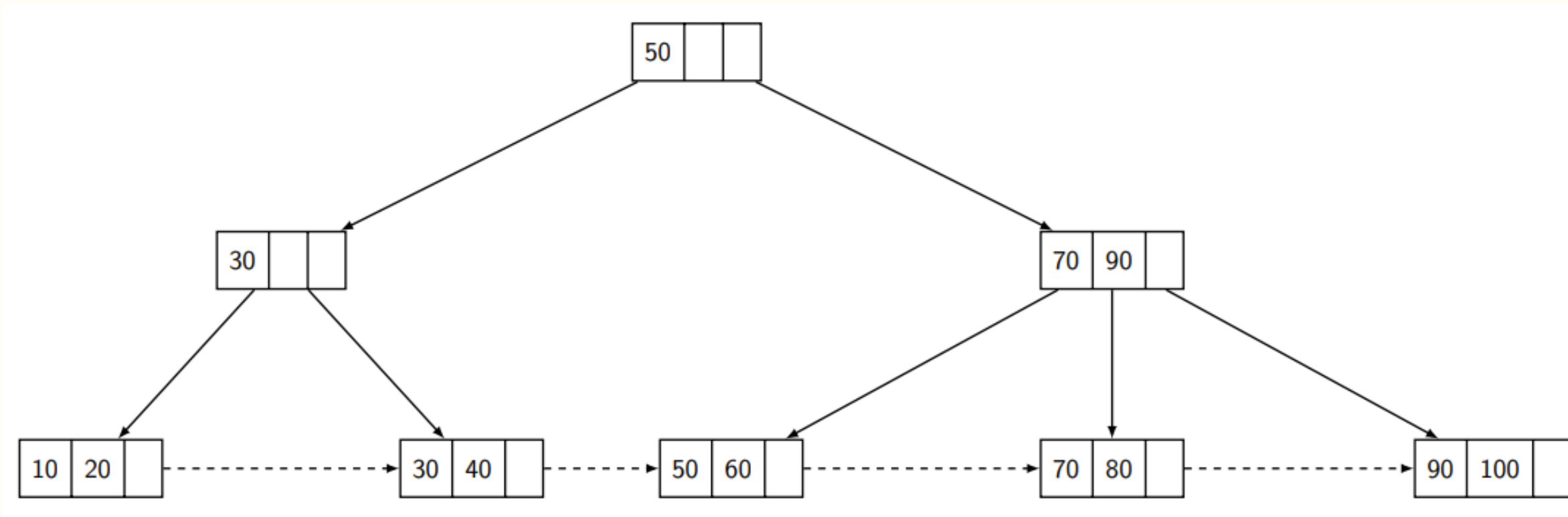
Inserindo o valor **100** (split)



Remoção

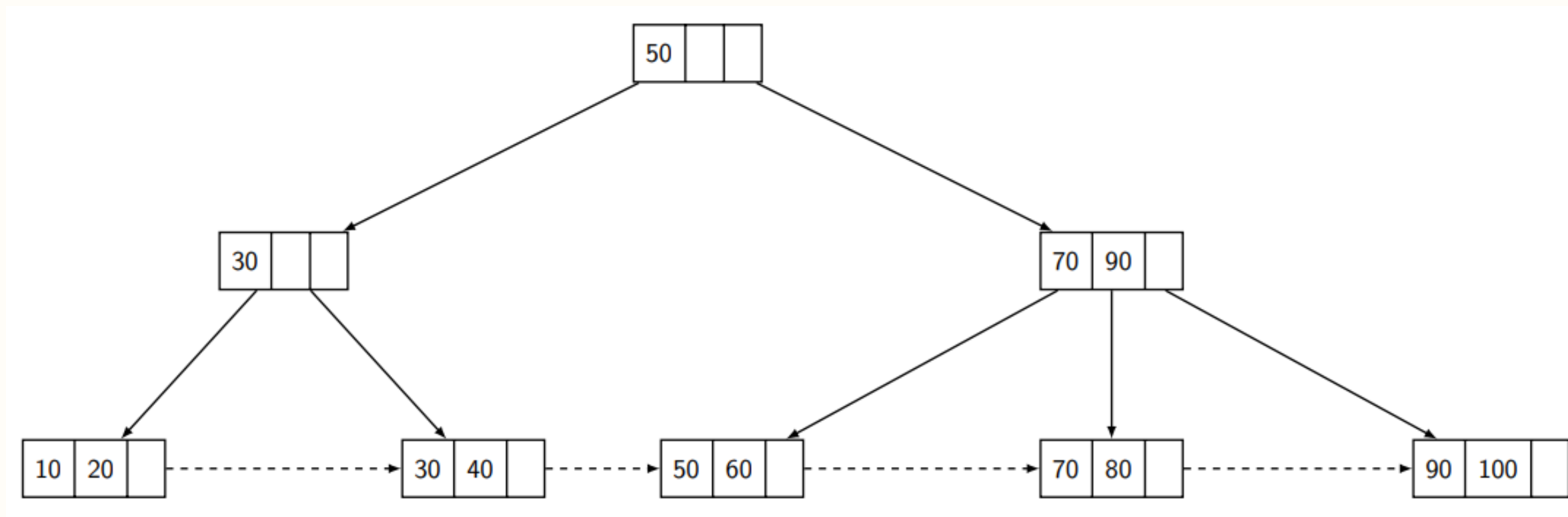
Remoção:

Árvore Inicial



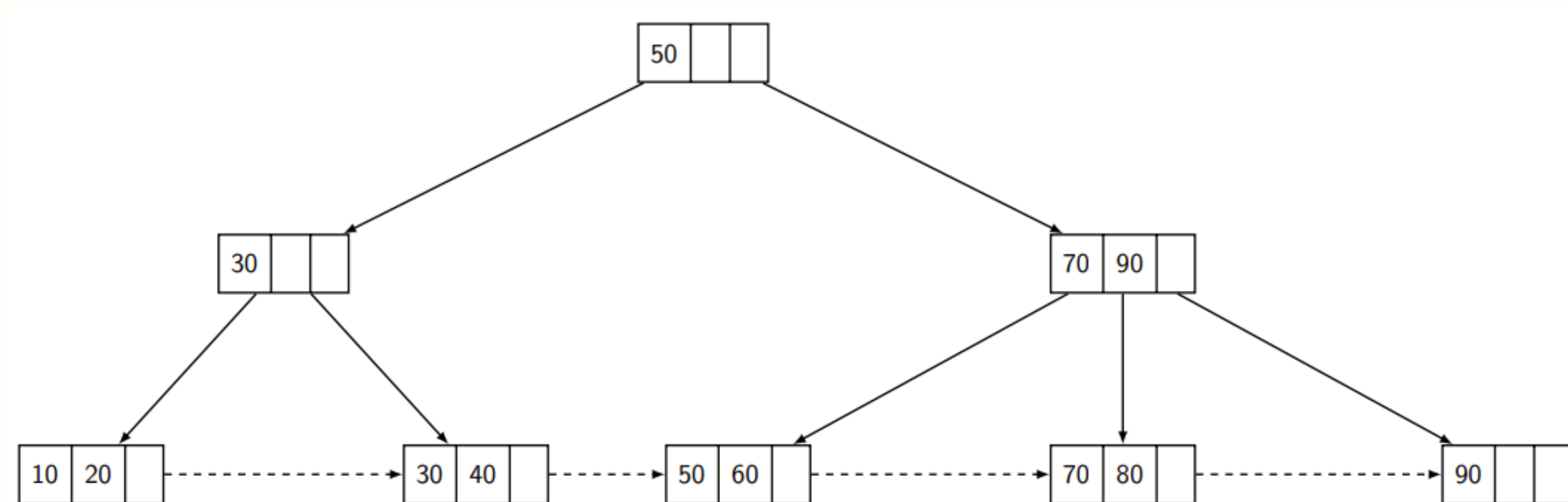
Remoção:

Removendo o valor **100**



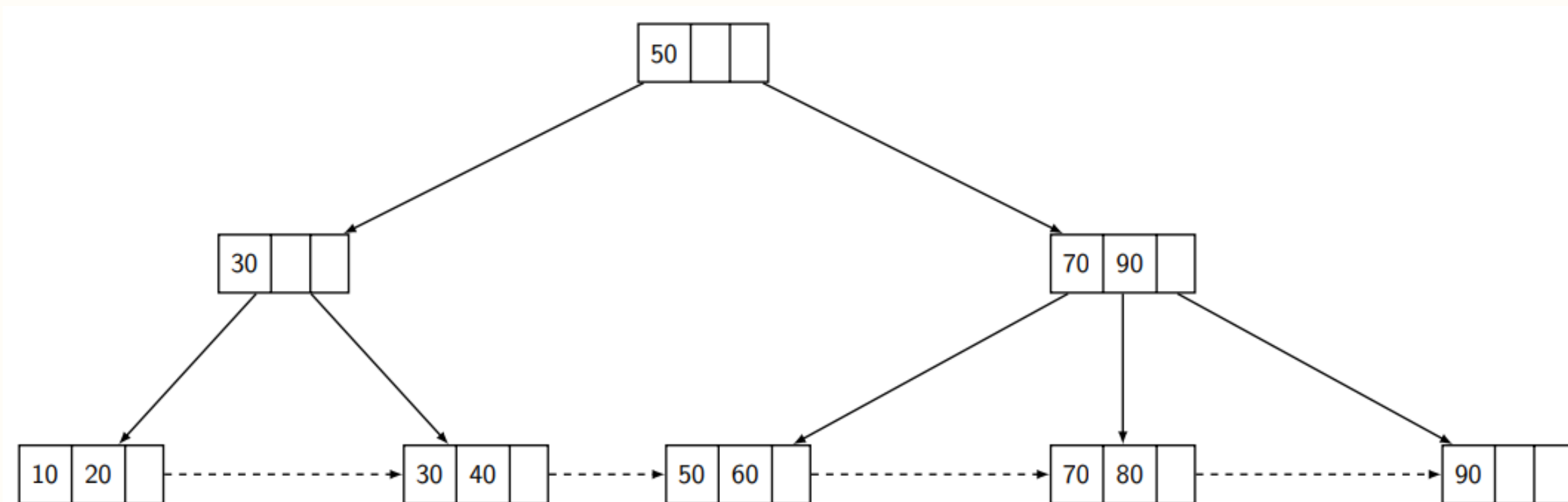
Remoção:

Removendo o valor **100**



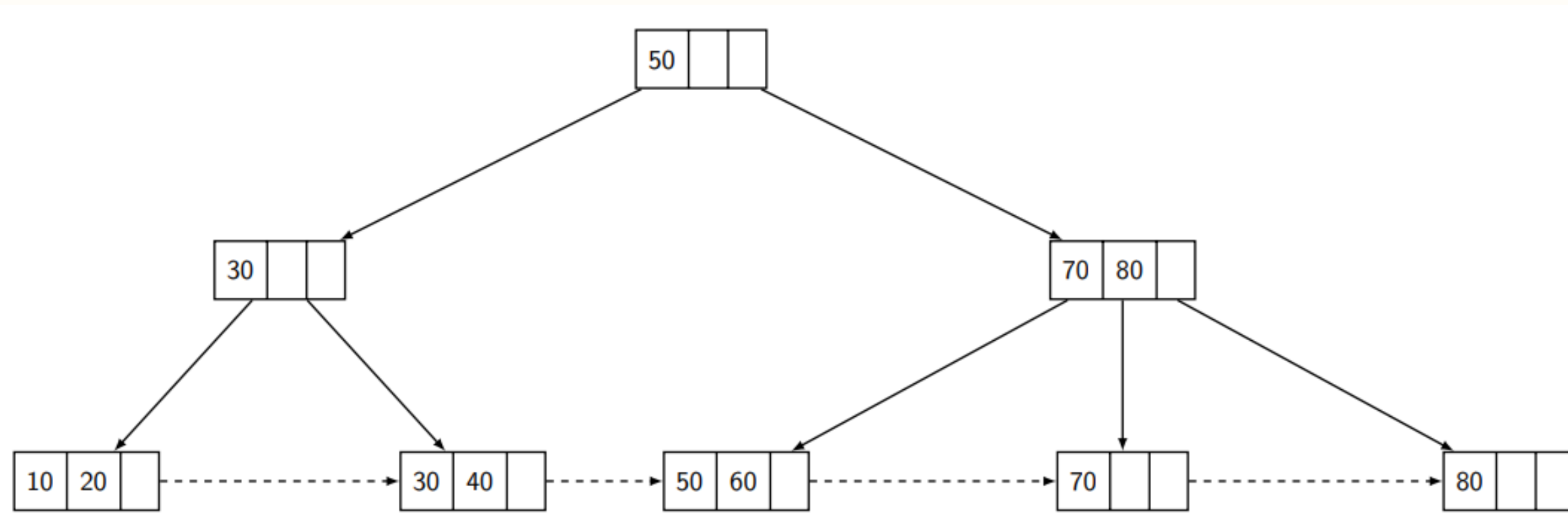
Remoção:

Removendo o valor **90**



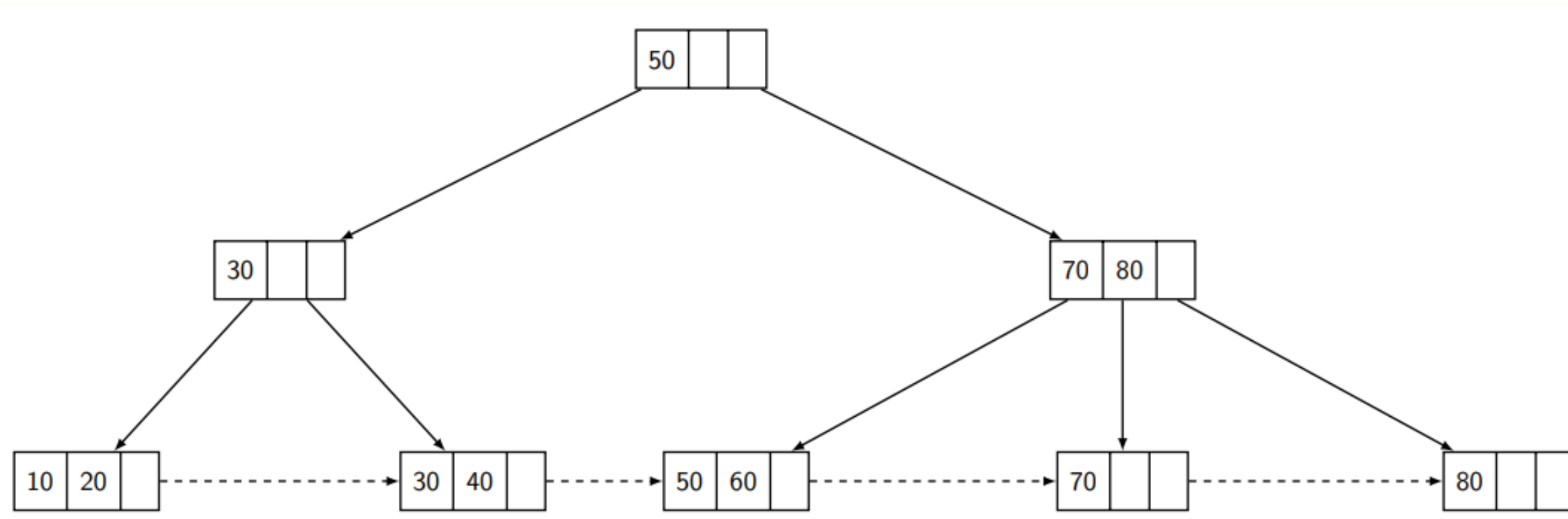
Remoção:

Removendo o valor **90** (rotação)



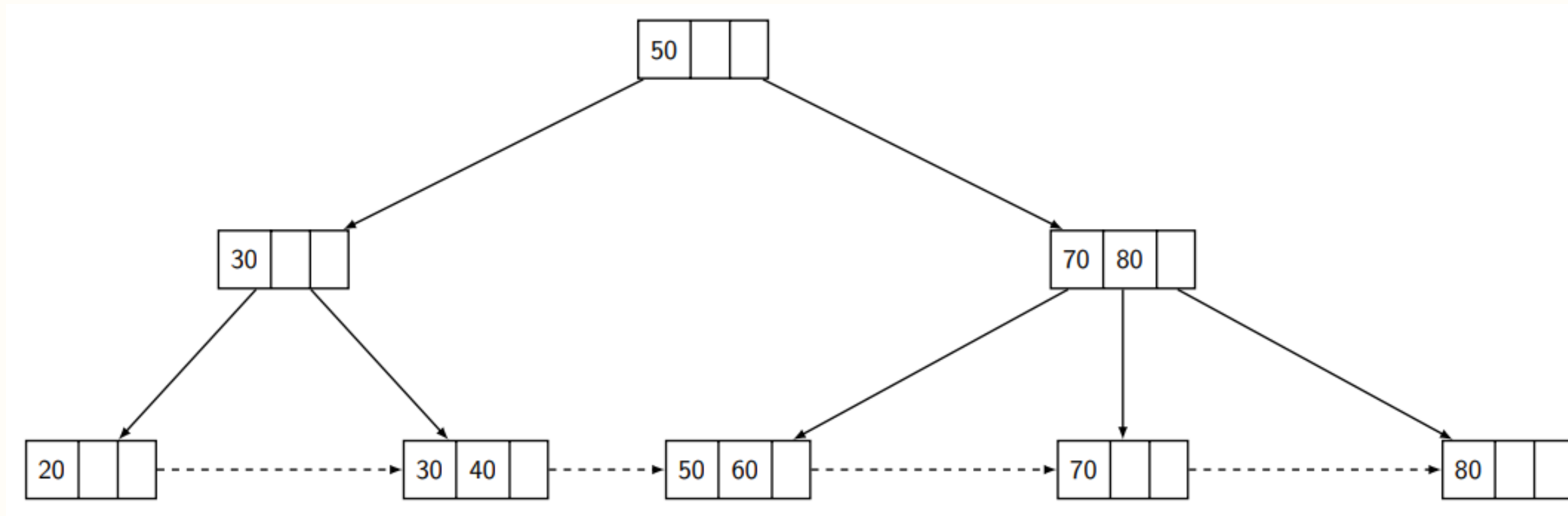
Remoção:

Removendo o valor **10**



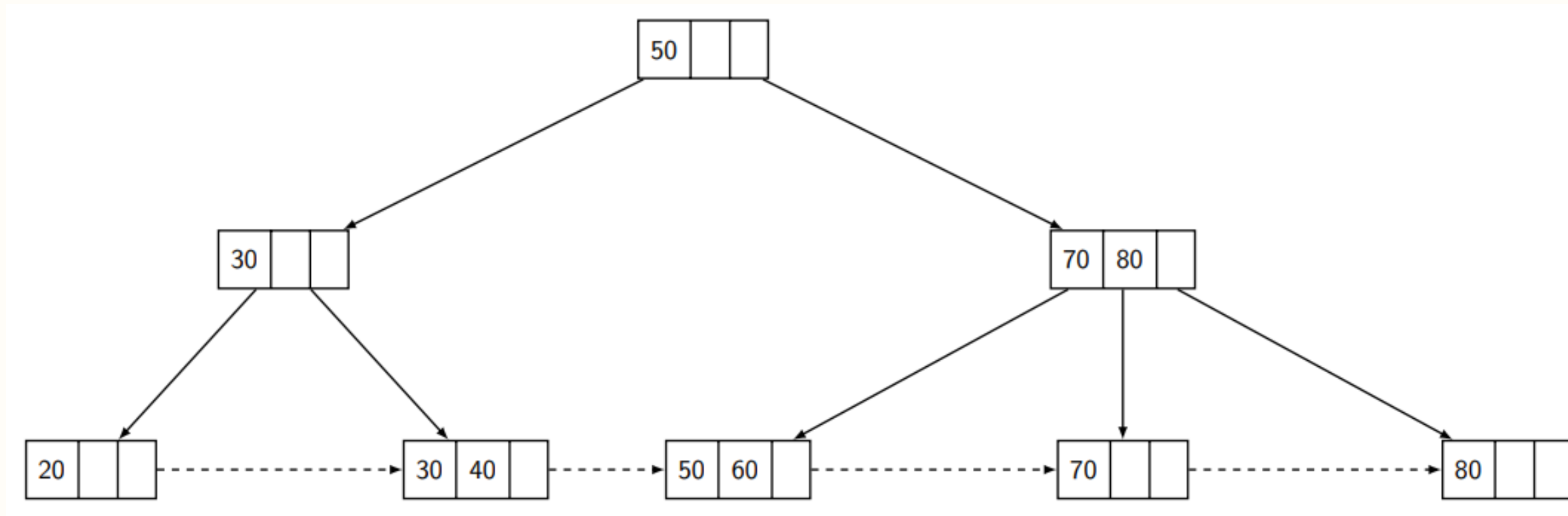
Remoção:

Removendo o valor **10**



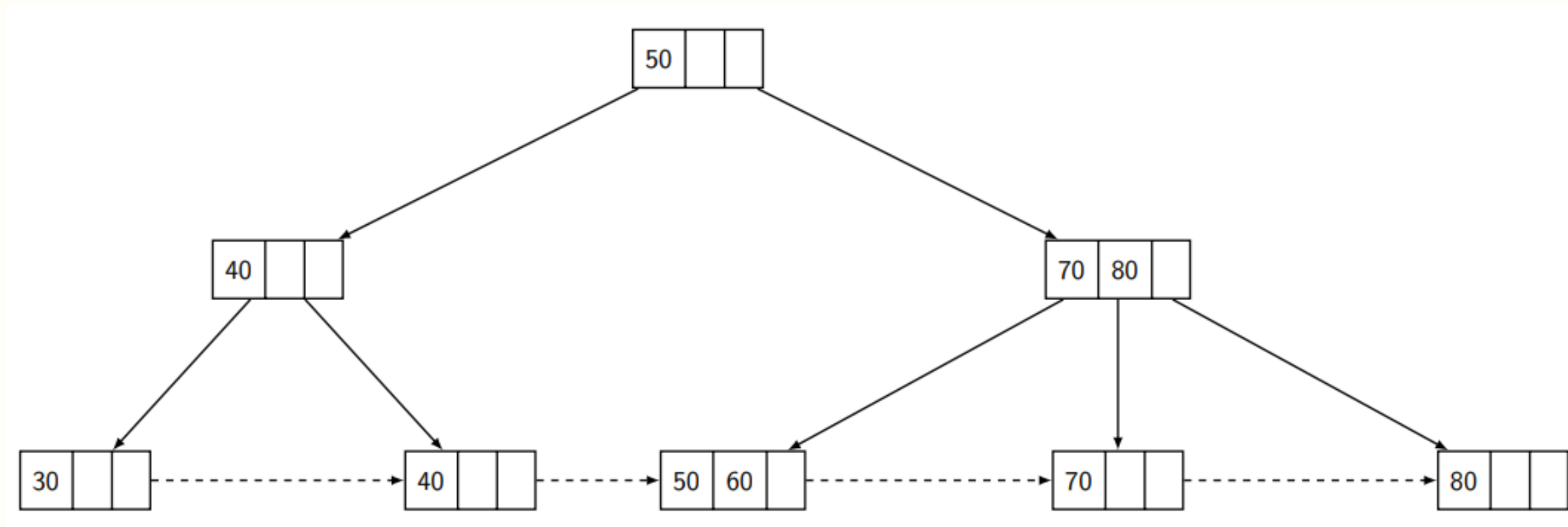
Remoção:

Removendo o valor **20**



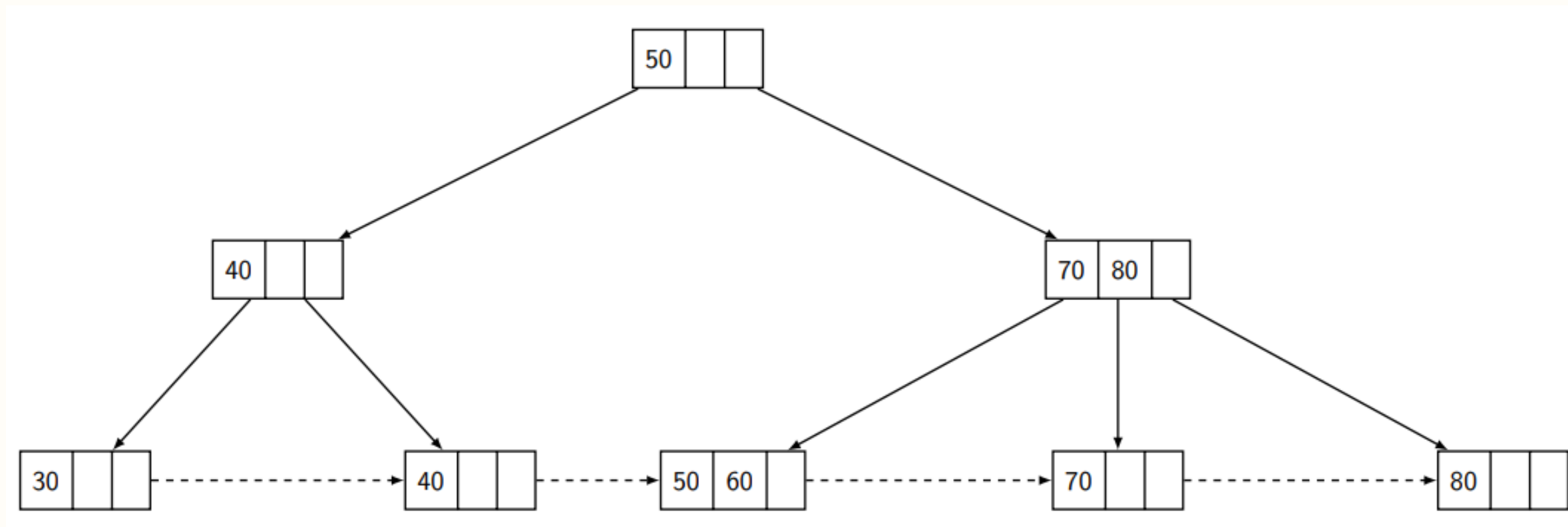
Remoção:

Removendo o valor **20** (rotação)



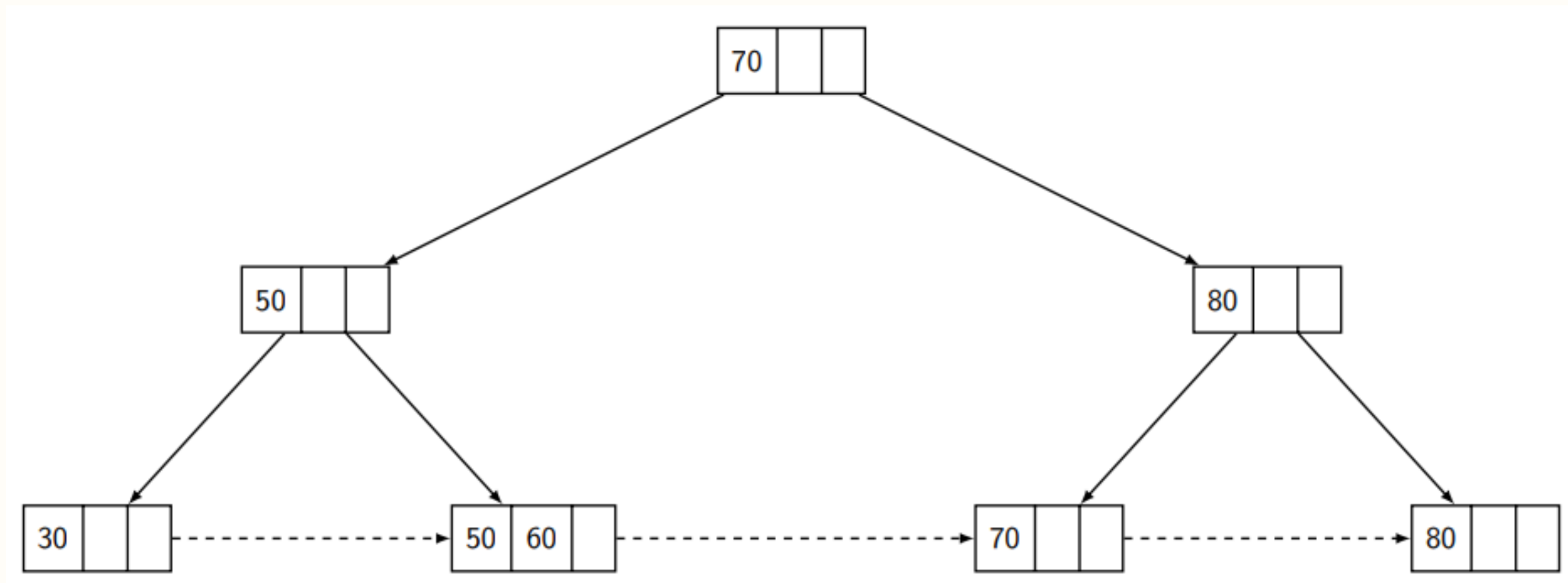
Remoção:

Removendo o valor **40**



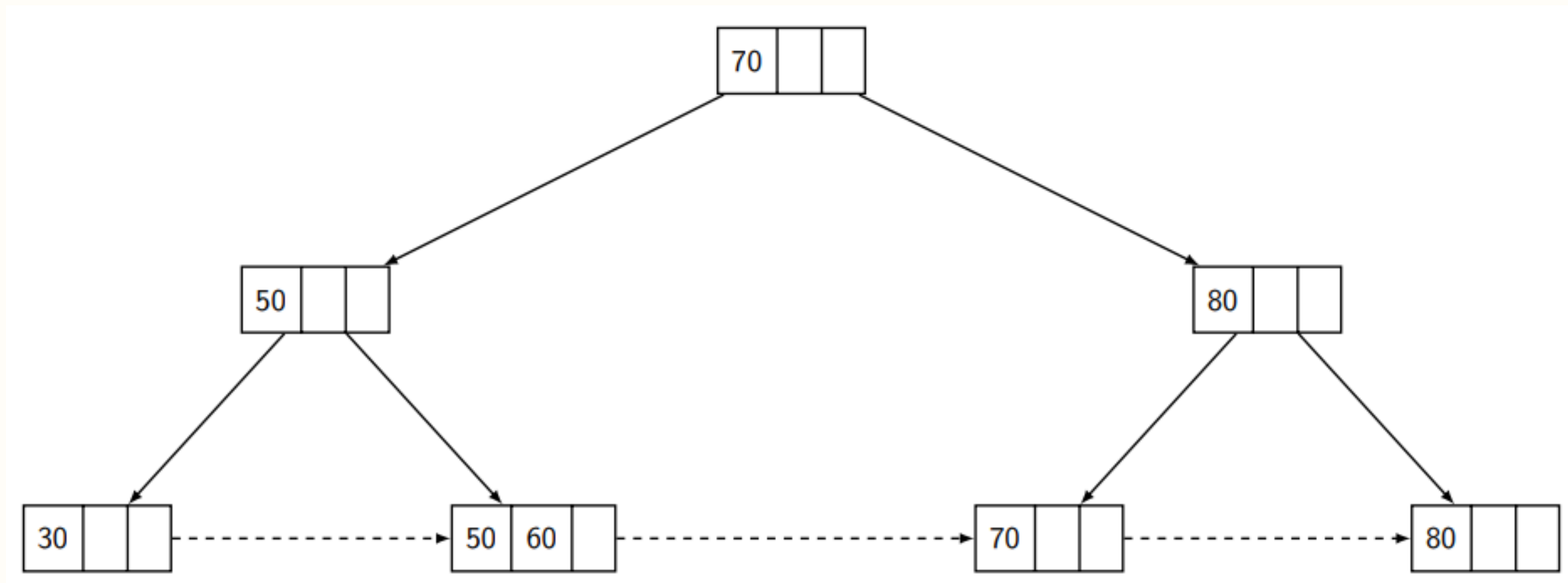
Remoção:

Removendo o valor **40**



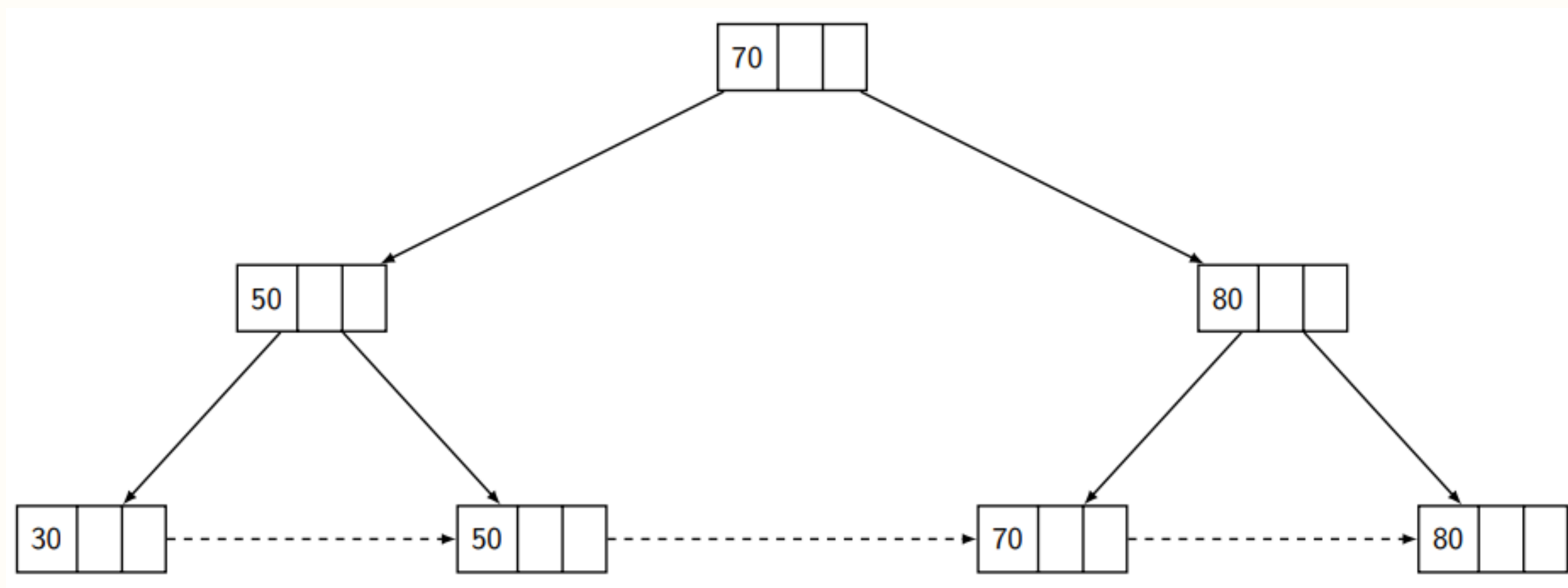
Remoção:

Removendo o valor **60**



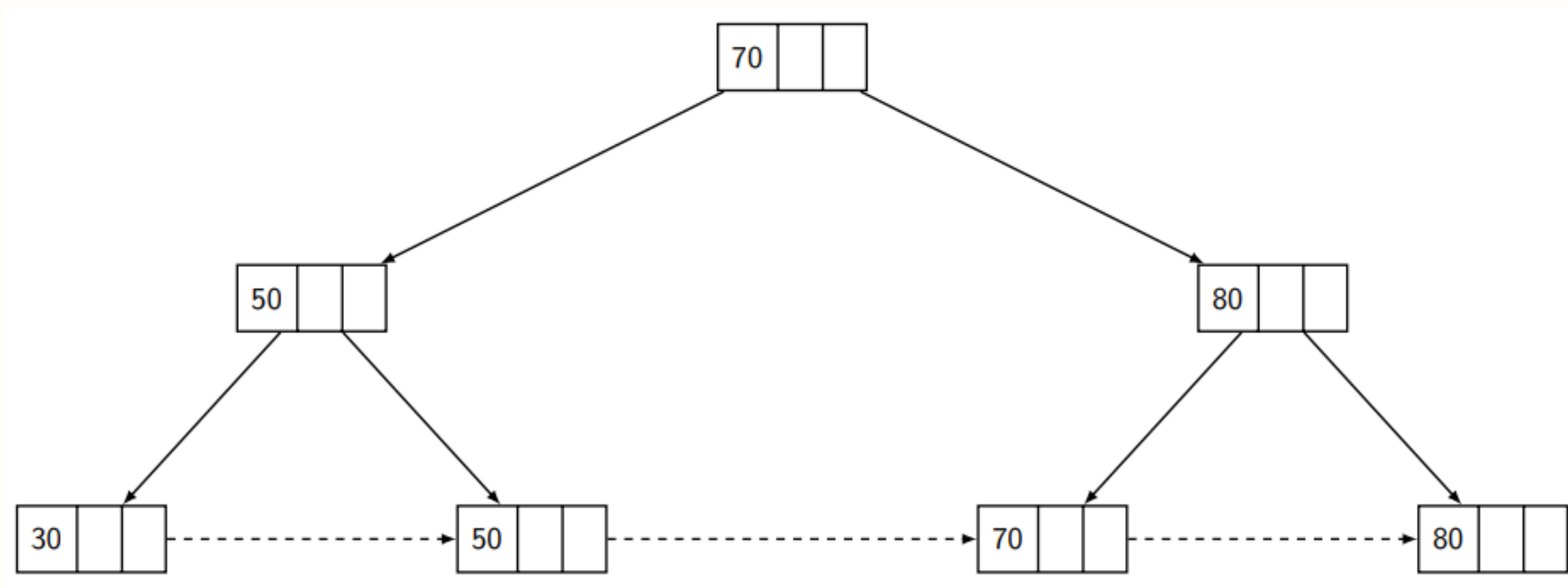
Remoção:

Removendo o valor **60**



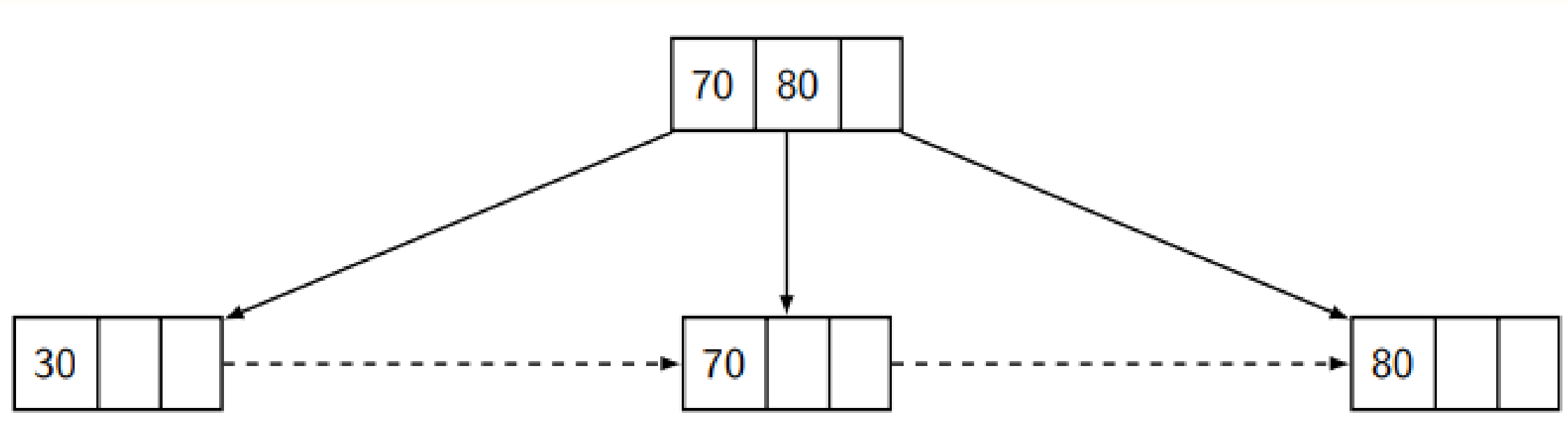
Remoção:

Removendo o valor **50**



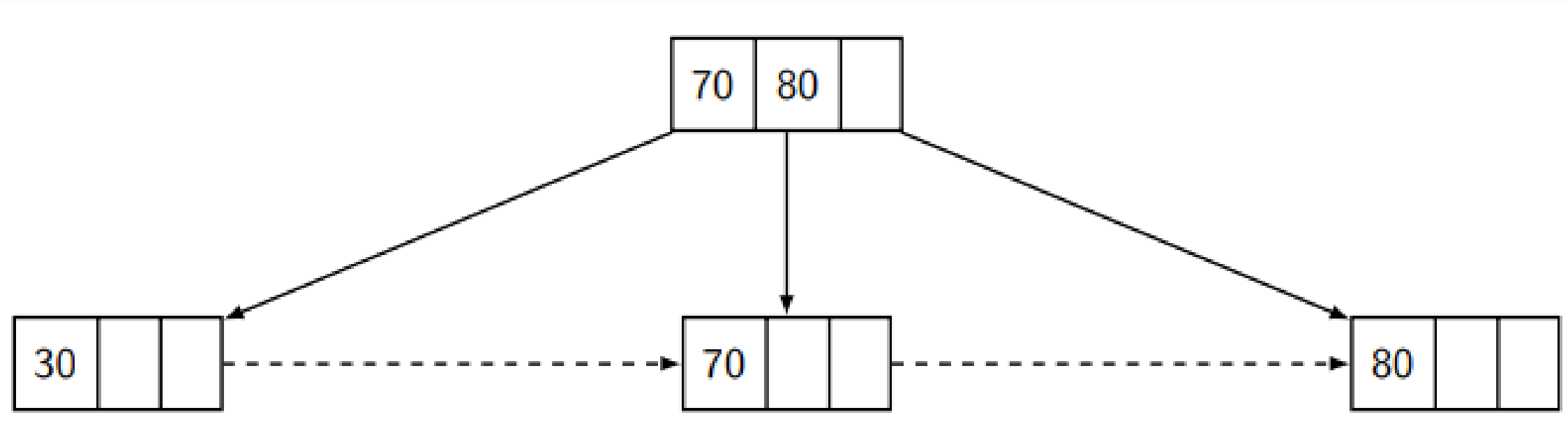
Remoção:

Removendo o valor **50**



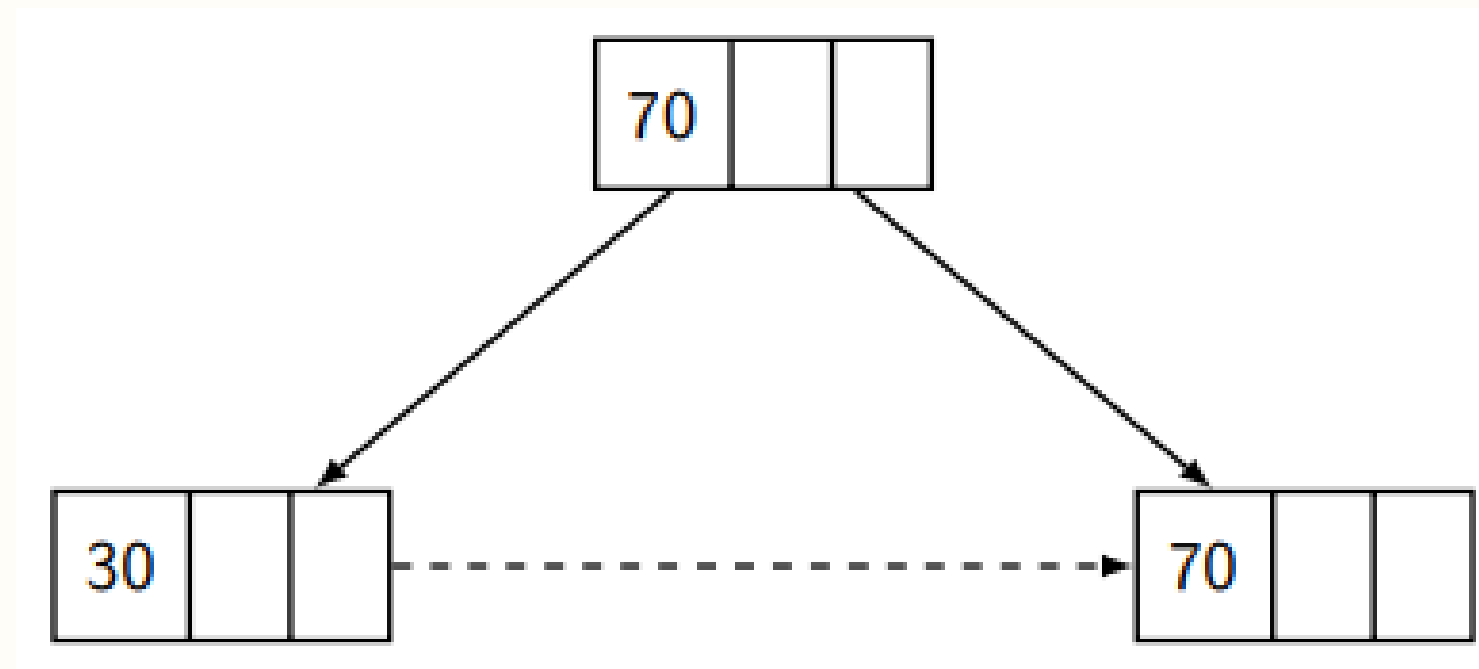
Remoção:

Removendo o valor **80**



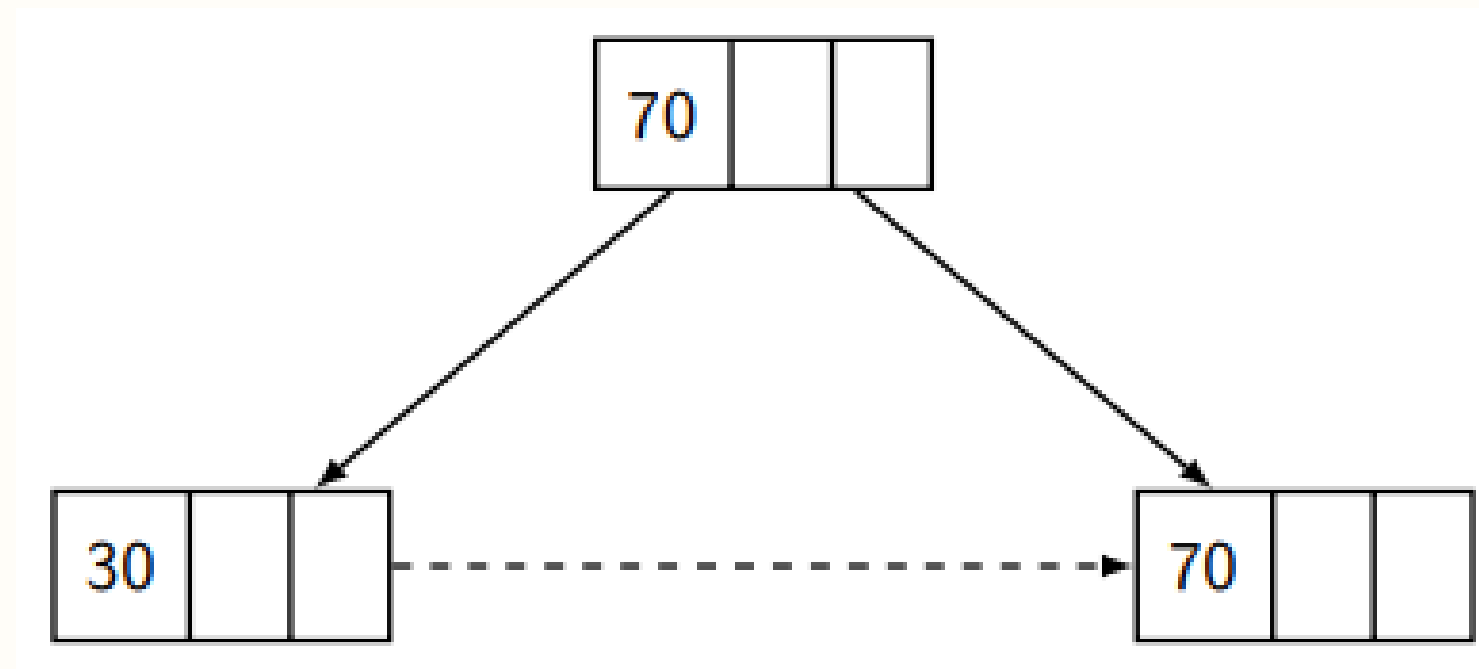
Remoção:

Removendo o valor **80**



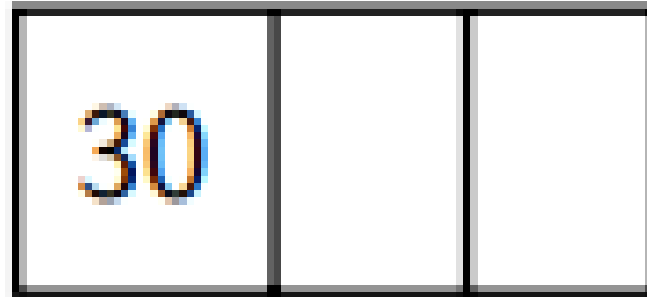
Remoção:

Removendo o valor **70**



Remoção:

Removendo o valor **70**

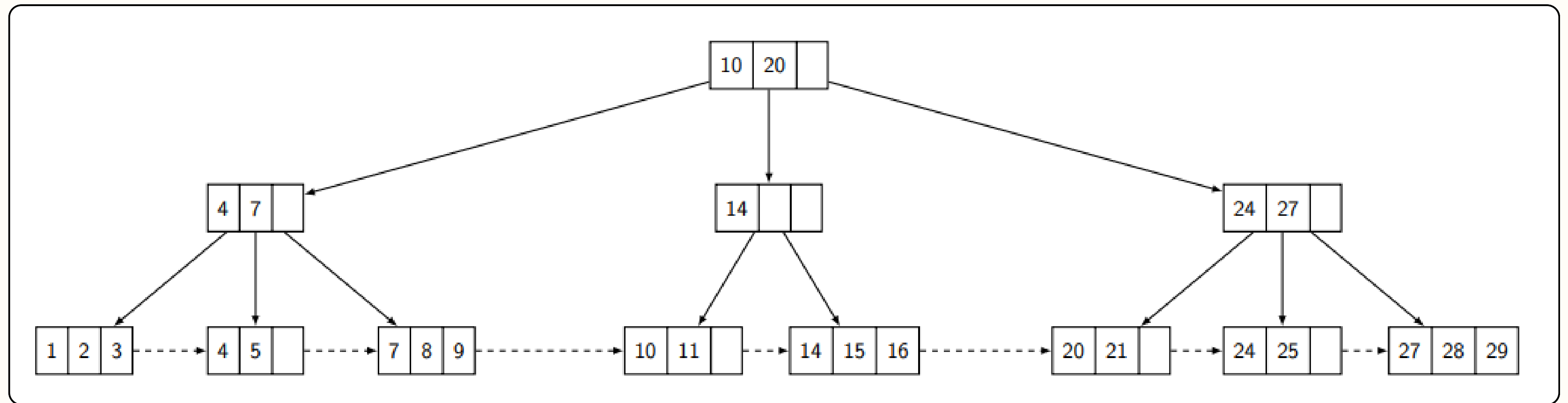


Busca em Intervalo

(Range Query)

Busca em Intervalo:

Queremos o intervalo [7, 25]



Aplicações

Aplicações:

Sistemas de Arquivos

ReiserFS, NSS, XFS, JFS, ReFS e BFS utilizam essa estrutura para indexação de metadados

Aplicações:

Bancos de Dados

IBM Db2, Informix, Microsoft SQL Server, Oracle, Sybase ASE e SQLite



ORACLE



Aplicações:

Memoria Não Volátil

Adotada em sistemas de memória de acesso aleatório não volátil (NVRAM)

Exemplos

Exemplos:

Exemplo 1 - Tamanho de t: O tamanho de t é definido pelo tamanho da página do disco, como dito anteriormente, usaremos a inequação a seguir e alguns valores pre definidos para encontrarmos um valor de t.

$$(n.K) + ((n + 1).V) + M \leq P$$

Onde:

- P é o tamanho da página. Usaremos $P = 4096$ bytes (padrão Windows/Linux);
- K é o tamanho da chave. Usaremos $K = 8$ bytes (tamanho de um inteiro de 64 bits);
- V é o tamanho do ponteiro. Usaremos $V = 8$ bytes;
- M são os metadados, ou seja, as informações armazenadas no nó.
- n é o número máximo de chaves que queremos descobrir.

Exemplos:

$$(n.K) + ((n + 1).V) + M \leq P$$

Exemplos:

$$(n.K) + ((n + 1).V) + M \leq P$$
$$(n.8) + ((n + 1).8) + 16 \leq 4096$$

Exemplos:

$$(n.K) + ((n + 1).V) + M \leq P$$

$$(n.8) + ((n + 1).8) + 16 \leq 4096$$

$$8n + 8n + 8 + 16 \leq 4096$$

Exemplos:

$$(n.K) + ((n + 1).V) + M \leq P$$

$$(n.8) + ((n + 1).8) + 16 \leq 4096$$

$$8n + 8n + 8 + 16 \leq 4096$$

$$16n + 24 \leq 4096$$

Exemplos:

$$(n.K) + ((n + 1).V) + M \leq P$$

$$(n.8) + ((n + 1).8) + 16 \leq 4096$$

$$8n + 8n + 8 + 16 \leq 4096$$

$$16n + 24 \leq 4096$$

$$16n \leq 4072$$

Exemplos:

$$(n.K) + ((n + 1).V) + M \leq P$$

$$(n.8) + ((n + 1).8) + 16 \leq 4096$$

$$8n + 8n + 8 + 16 \leq 4096$$

$$16n + 24 \leq 4096$$

$$16n \leq 4072$$

$$n = 254$$

Exemplos:

$$2t - 1 = n$$

Exemplos:

$$2t - 1 = n$$

$$2t - 1 = 254$$

Exemplos:

$$2t - 1 = n$$

$$2t - 1 = 254$$

$$2t = 255$$

Exemplos:

$$2t - 1 = n$$

$$2t - 1 = 254$$

$$2t = 255$$

$$t = 127$$

Exemplos:

Exemplo 2 - Custo de uma Busca em Intervalo: Vamos avaliar a complexidade de tempo de uma busca por intervalo (range query)

Exemplos:

Exemplo 3 - Split Preventivo: Por que fazer divisões preventivas dos nós? e qual o rumo da chave mediana de uma folha? e de um nó interno?

Referências:

Bayer, R. and McCreight, E. M. (1972). Organization and maintenance of large ordered indices. *Acta Informatica*, 1(3):173–189.

Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley, 2 edition.

Comer, D. (1979). The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137

Apple Inc. (2020). Apple File System Reference. Disponível em: <https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>.

Dharamjeet, Chen, T.-Y., Chang, Y.-H., Wu, C.-F., Lee, C.-H., and Shih, W.-K. (2021). Beyond write-reduction consideration: A wear-leveling-enabled B+-tree indexing scheme over an NVRAM-based architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(12):2455–2466.

Giampaolo, D. (1999). *Practical File System Design with the Be File System*. Morgan Kaufmann.

Jagadish, H. V., Ooi, B. C., Tan, K.-L., Yu, C., and Zhang, R. (2005). iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *ACM Transactions on Database Systems*, 30(2):364–397.

Ramakrishnan, R. and Gehrke, J. (2003). *Database Management Systems*. McGraw-Hill, 3 edition.